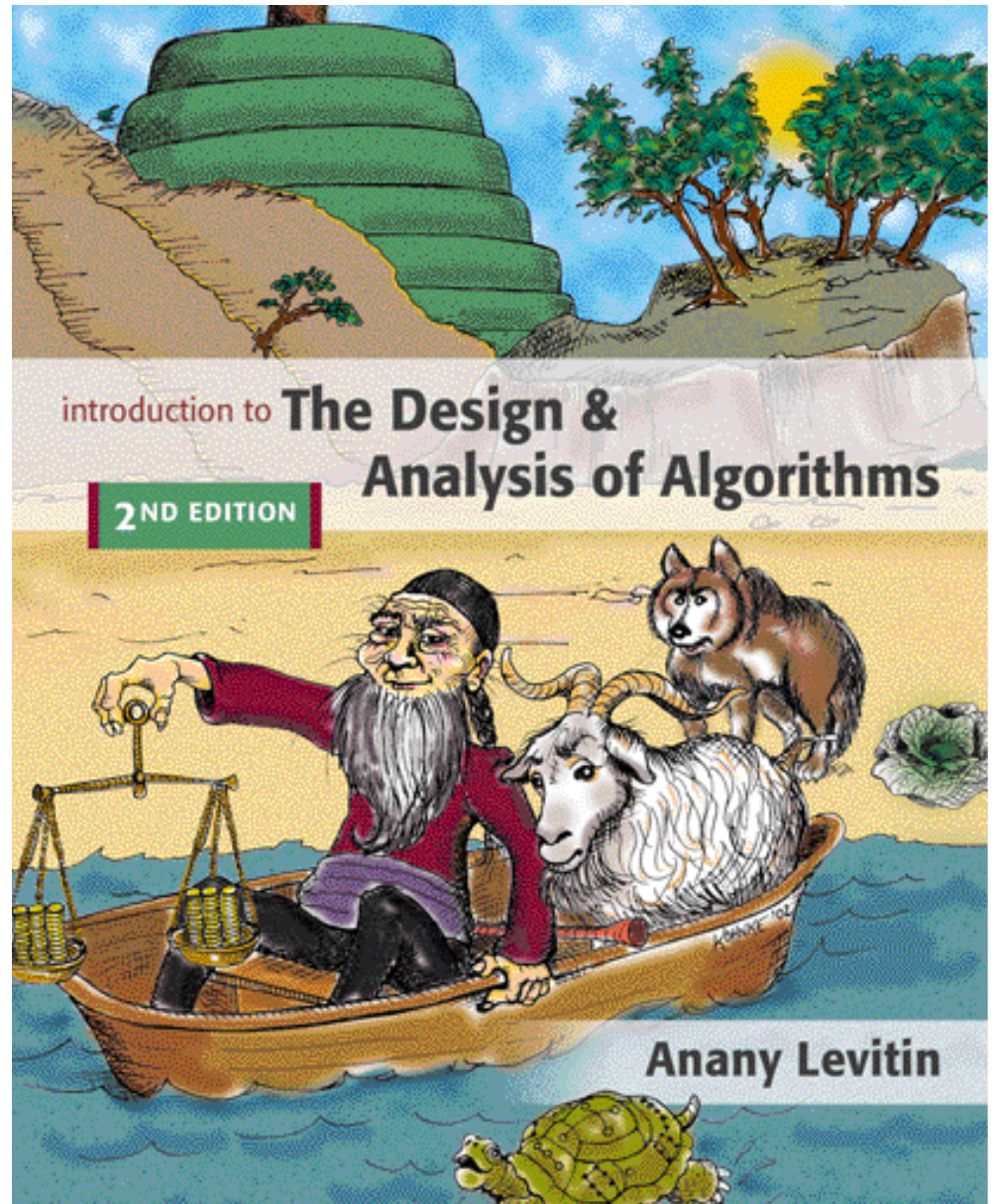


Chapter 4

Decrease-and-Conquer



Decrease-and-Conquer

1. Reduce problem instance to smaller instance of the same problem
 2. Solve smaller instance
 3. Extend solution of smaller instance to obtain solution to original instance
- Can be implemented either top-down (recursively) or bottom-up (without a recursion)
 - Also referred to as *inductive* or *incremental* approach

3 Types of Decrease and Conquer

- Decrease by a constant (usually by 1):
 - insertion sort
 - graph traversal algorithms (DFS and BFS)
 - topological sorting
 - algorithms for generating permutations, subsets
- Decrease by a constant factor (usually by half)
 - binary search and bisection method
 - exponentiation by squaring
 - multiplication à la russe
- Variable-size decrease
 - Euclid's algorithm
 - selection by partition
 - Nim-like games

What's the difference?

Consider the problem of exponentiation: Compute a^n

- Brute Force:
- Divide and conquer:
- Decrease by one:
- Decrease by constant factor:

What's the difference?

Consider the problem of exponentiation: Compute a^n

- Brute Force:
- Divide and conquer:
- Decrease by one:
- Decrease by constant factor:

Insertion Sort

To sort array $A[0..n-1]$, sort $A[0..n-2]$ recursively and then insert $A[n-1]$ in its proper place among the sorted $A[0..n-2]$

- Usually implemented bottom up (nonrecursively)

Example: Sort 6, 4, 1, 8, 5

6		<u>4</u>	1	8	5
4	6		<u>1</u>	8	5
1	4	6		<u>8</u>	5
1	4	6	8		<u>5</u>
1	4	5	6	8	

Pseudocode of Insertion Sort

ALGORITHM *InsertionSort*($A[0..n - 1]$)

//Sorts a given array by insertion sort

//Input: An array $A[0..n - 1]$ of n orderable elements

//Output: Array $A[0..n - 1]$ sorted in nondecreasing order

for $i \leftarrow 1$ **to** $n - 1$ **do**

$v \leftarrow A[i]$

$j \leftarrow i - 1$

while $j \geq 0$ **and** $A[j] > v$ **do**

$A[j + 1] \leftarrow A[j]$

$j \leftarrow j - 1$

$A[j + 1] \leftarrow v$

Analysis of Insertion Sort

- Time efficiency

$$C_{worst}(n) = n(n-1)/2 \in \Theta(n^2)$$

$$C_{avg}(n) \approx n^2/4 \in \Theta(n^2)$$

$$C_{best}(n) = n - 1 \in \Theta(n) \text{ (also fast on almost sorted arrays)}$$

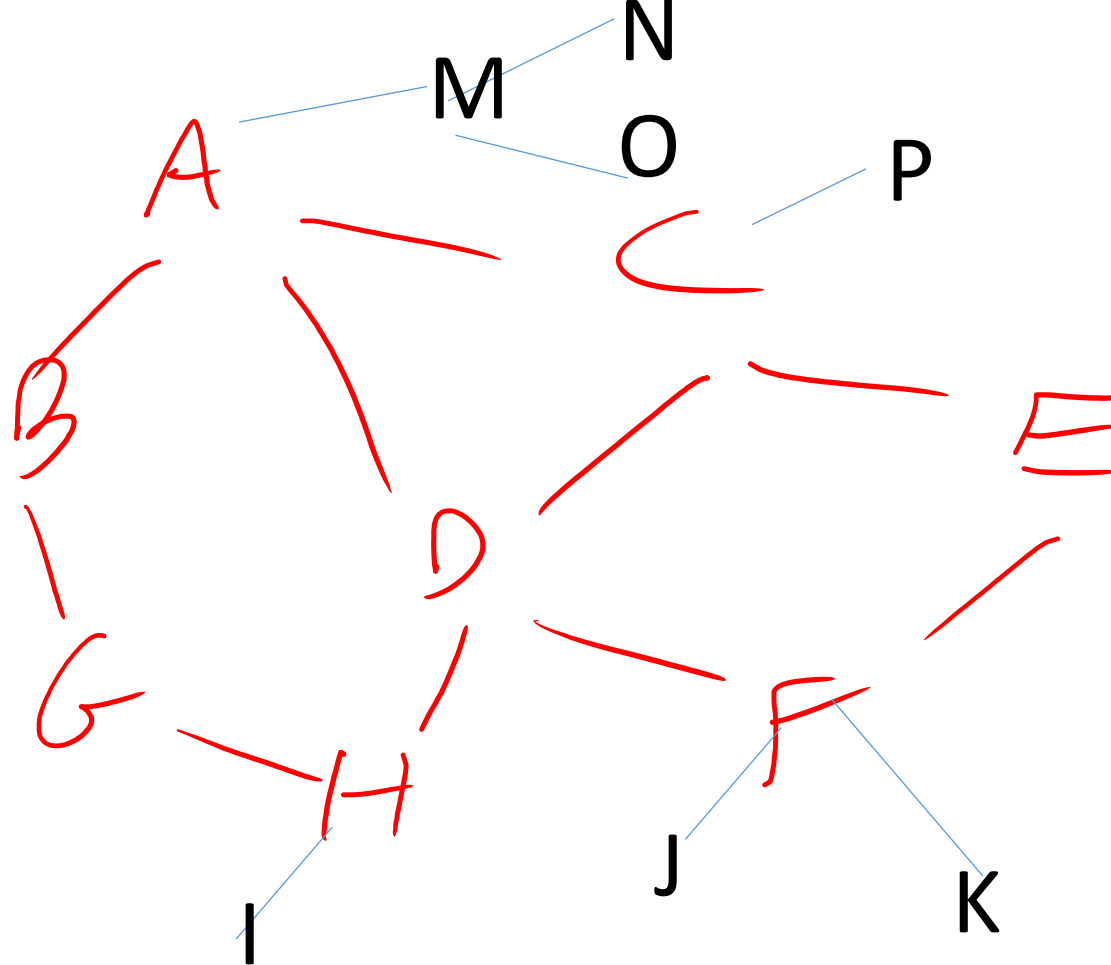
- Space efficiency: in-place
- Stability: yes
- Best elementary sorting algorithm overall
- Binary insertion sort

Graph Traversal

Many problems require processing all graph vertices (and edges) in systematic fashion

Graph traversal algorithms:

- Depth-first search (DFS)
- Breadth-first search (BFS)



Depth-First Search (DFS)

- Visits graph's vertices by always moving away from last visited vertex to unvisited one, backtracks if no adjacent unvisited vertex is available.
- Uses a stack
 - a vertex is pushed onto the stack when it's reached for the first time
 - a vertex is popped off the stack when it becomes a dead end, i.e., when there is no adjacent unvisited vertex
- “Redraws” graph in tree-like fashion (with tree edges and back edges for undirected graph)

Pseudocode of DFS

ALGORITHM *DFS*(*G*)

//Implements a depth-first search traversal of a given graph

//Input: Graph $G = \langle V, E \rangle$

//Output: Graph *G* with its vertices marked with consecutive integers

//in the order they've been first encountered by the DFS traversal

mark each vertex in *V* with 0 as a mark of being “unvisited”

count $\leftarrow$ 0

for each vertex *v* in *V* **do**

if *v* is marked with 0

dfs(*v*)

dfs(*v*)

//visits recursively all the unvisited vertices connected to vertex *v* by a path

//and numbers them in the order they are encountered

//via global variable *count*

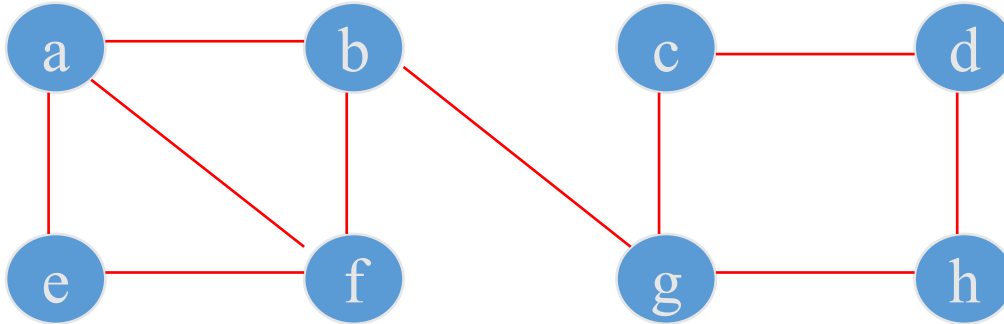
count $\leftarrow$ *count* + 1; mark *v* with *count*

for each vertex *w* in *V* adjacent to *v* **do**

if *w* is marked with 0

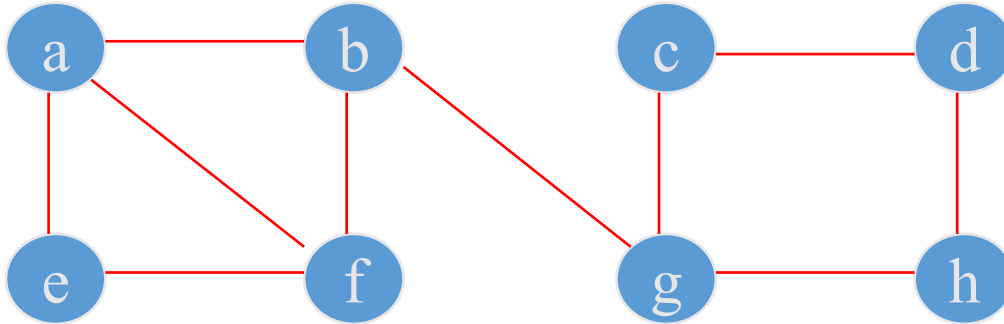
dfs(*w*)

Example: DFS traversal of undirected graph



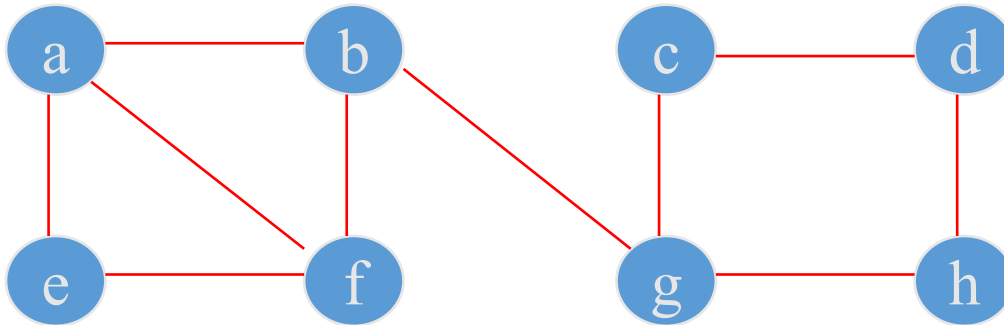
DFS tree:

Example: DFS traversal of undirected graph



DFS tree:

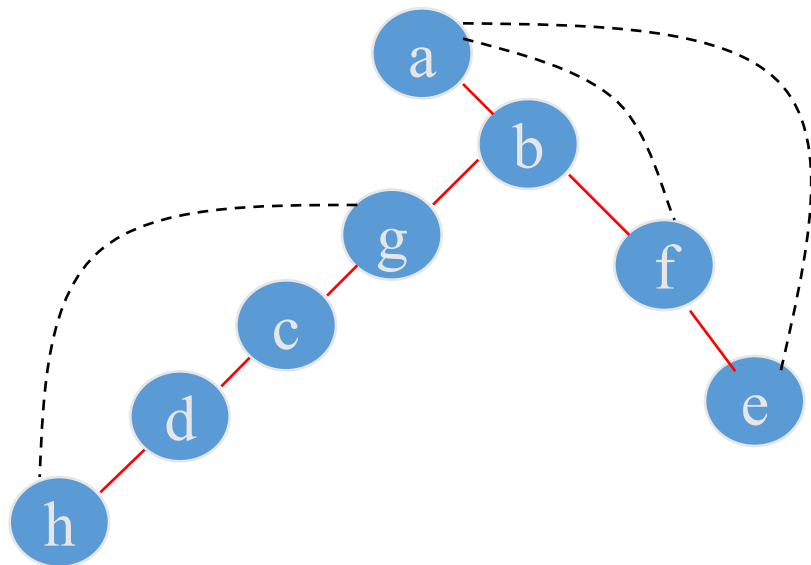
Example: DFS traversal of undirected graph



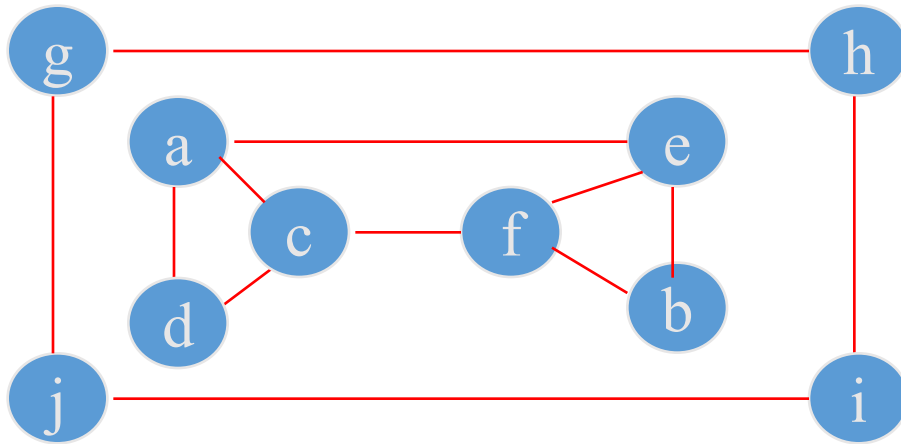
DFS traversal stack:

$h_{8,3}$
 $d_{7,4}$
 $c_{6,5}$
 $g_{5,6}$
 $e_{4,1}$
 $f_{3,2}$
 $b_{2,7}$
 $a_{1,8}$

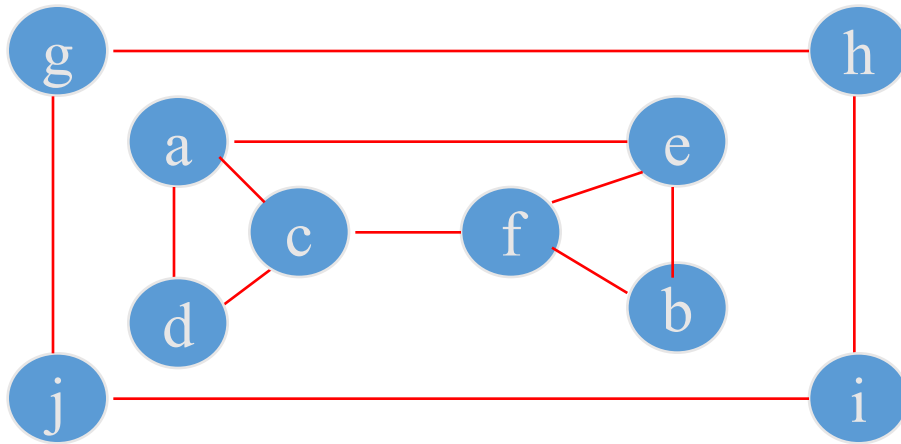
DFS tree:



DFS traversal of undirected graph



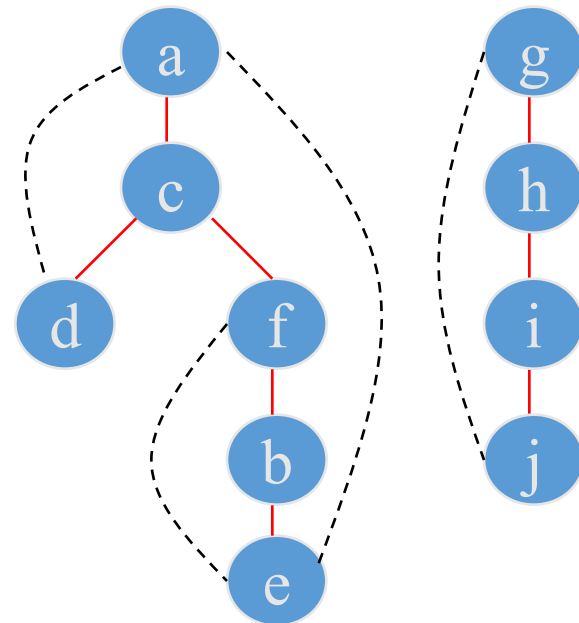
DFS traversal of undirected graph



DFS traversal stack:

	$e_{6,2}$	
	$b_{5,3}$	$j_{10,7}$
$d_{3,1}$	$f_{4,4}$	$i_{9,8}$
$c_{2,5}$		$h_{8,9}$
$a_{1,6}$		$g_{7,10}$

DFS tree:



Notes on DFS

- DFS can be implemented with graphs represented as:
 - adjacency matrices: $\Theta(V^2)$
 - adjacency lists: $\Theta(|V| + |E|)$
- Yields two distinct ordering of vertices:
 - order in which vertices are first encountered (pushed onto stack)
 - order in which vertices become dead-ends (popped off stack)
- Applications:
 - checking connectivity, finding connected components
 - checking acyclicity
 - finding articulation points and biconnected components
 - searching state-space of problems for solution (AI)

Breadth-first search (BFS)

- Visits graph vertices by moving across to all the neighbors of last visited vertex
- Instead of a stack, BFS uses a queue
- Similar to level-by-level tree traversal
- “Redraws” graph in tree-like fashion (with tree edges and cross edges for undirected graph)

Pseudocode of BFS

ALGORITHM *BFS*(*G*)

//Implements a breadth-first search traversal of a given graph

//Input: Graph $G = \{V, E\}$

//Output: Graph G with its vertices marked with consecutive integers

//in the order they have been visited by the BFS traversal

mark each vertex in V with 0 as a mark of being “unvisited”

count $\leftarrow$ 0

for each vertex v in V **do**

if v is marked with 0

bfs(v)

bfs(v)

//visits all the unvisited vertices connected to vertex v by a path

//and assigns them the numbers in the order they are visited

//via global variable *count*

count $\leftarrow$ *count* + 1; mark v with *count* and initialize a queue with v

while the queue is not empty **do**

for each vertex w in V adjacent to the front vertex **do**

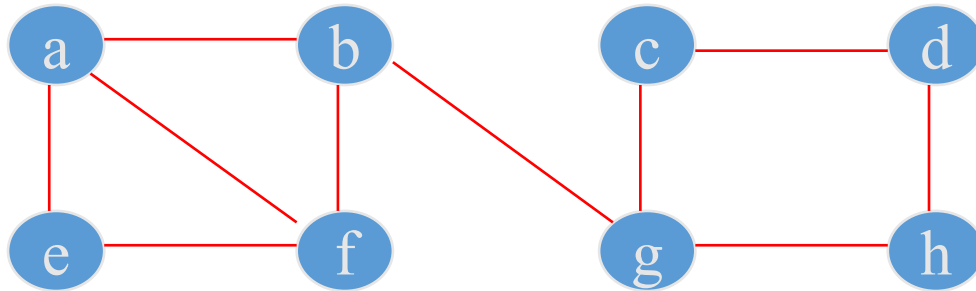
if w is marked with 0

count $\leftarrow$ *count* + 1; mark w with *count*

 add w to the queue

 remove the front vertex from the queue

Example of BFS traversal of undirected graph



BFS traversal queue:

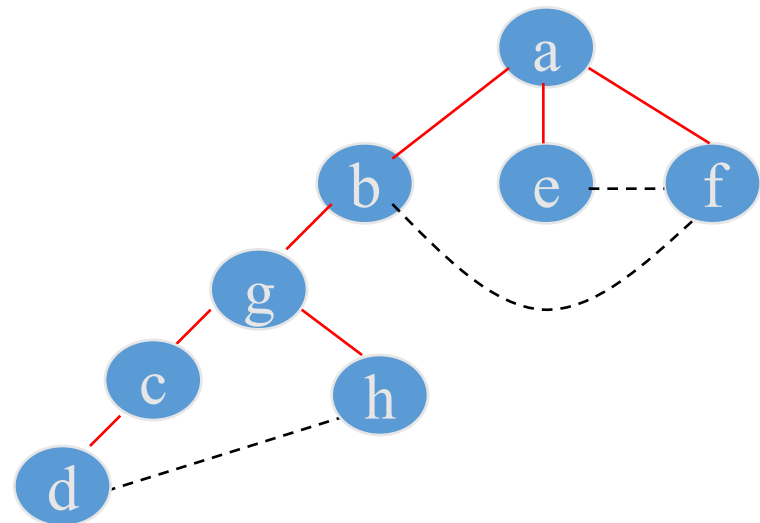
$a_1 b_2 e_3 f_4$

g_5

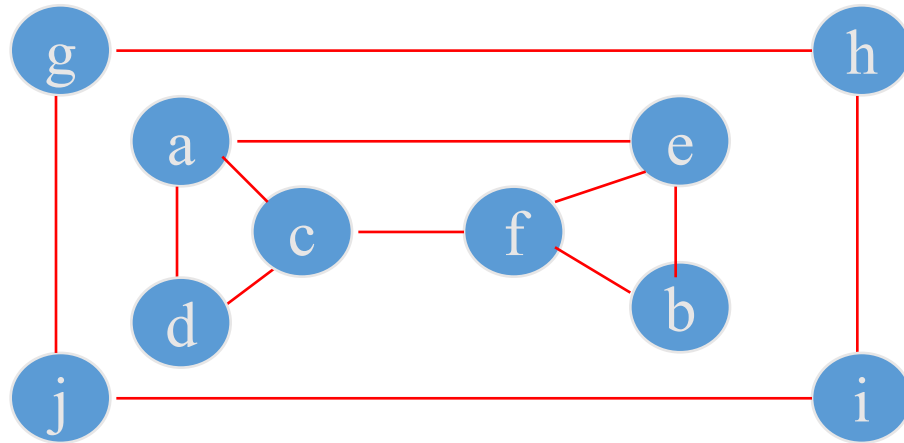
$c_6 h_7$

d_8

BFS tree:



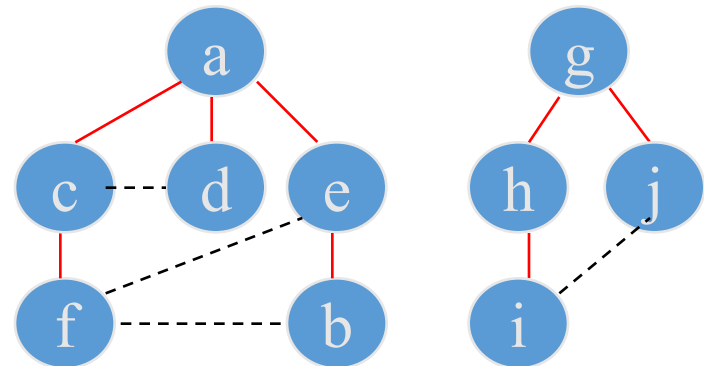
BFS traversal of undirected graph



**BFS traversal
queue:**

a₁ c₂ d₃ e₄
f₅ b₆
g₇ h₈ j₉
i₁₀

BFS tree:



Notes on BFS

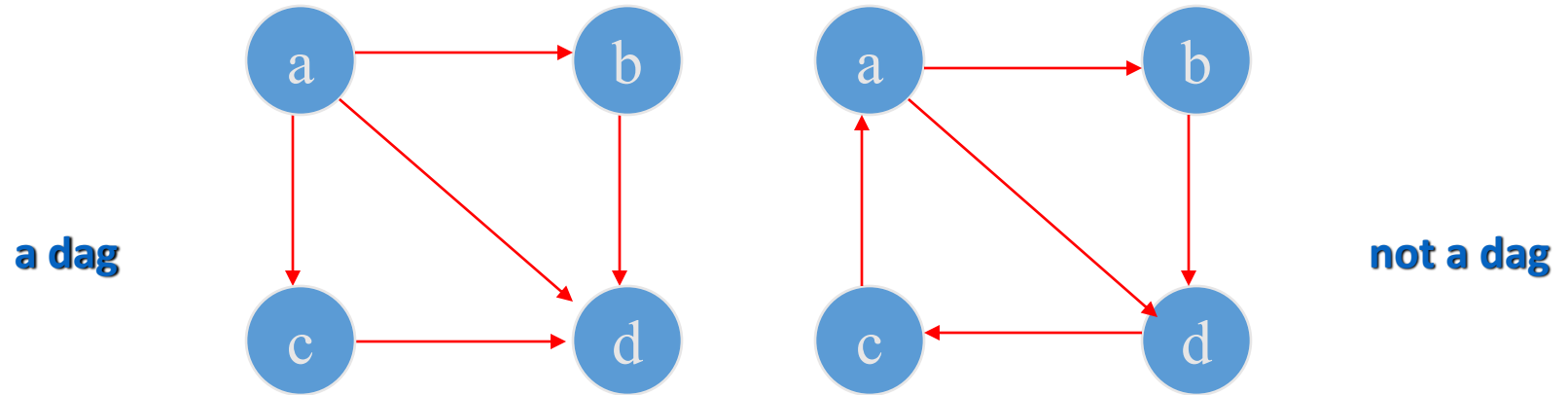
- BFS has same efficiency as DFS and can be implemented with graphs represented as:
 - adjacency matrices: $\Theta(V^2)$
 - adjacency lists: $\Theta(|V| + |E|)$
- Yields single ordering of vertices (order added/deleted from queue is the same)
- Applications: same as DFS, but can also find paths from a vertex to all other vertices with the smallest number of edges

Homework

- Exercise 4.2 on page 168
Question 1, 5

Dags and Topological Sorting

A dag: a directed acyclic graph, i.e. a directed graph with no (directed) cycles

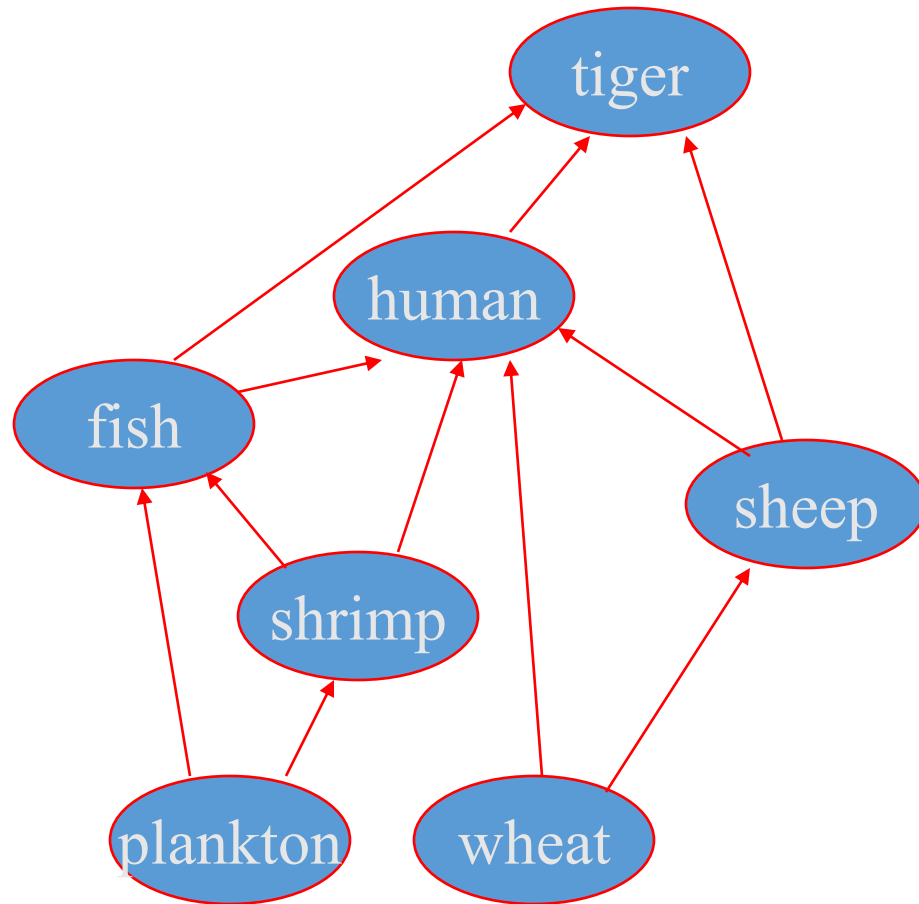


Arise in modeling many problems that involve prerequisite constraints (construction projects, document version control)

Vertices of a dag can be linearly ordered so that for every edge its starting vertex is listed before its ending vertex (topological sorting). Being a dag is also a necessary condition for topological sorting be possible.

Topological Sorting Example

Order the following items in a food chain



Topological Sorting Example

Course Registration:

There are five courses:

Course 3 requires course 1 and 2

Course 4 requires course 3

Course 5 requires course 4

In which order should student take the courses?

What is the graph?

Topological Sorting

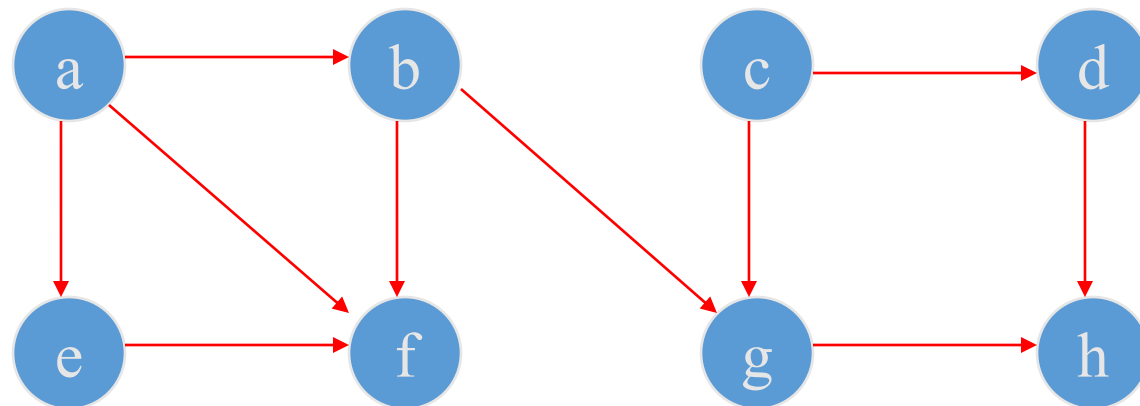
- Topological sorting can be solved by:
 - DFS-based algorithm
 - Source removal algorithm

DFS-based Algorithm

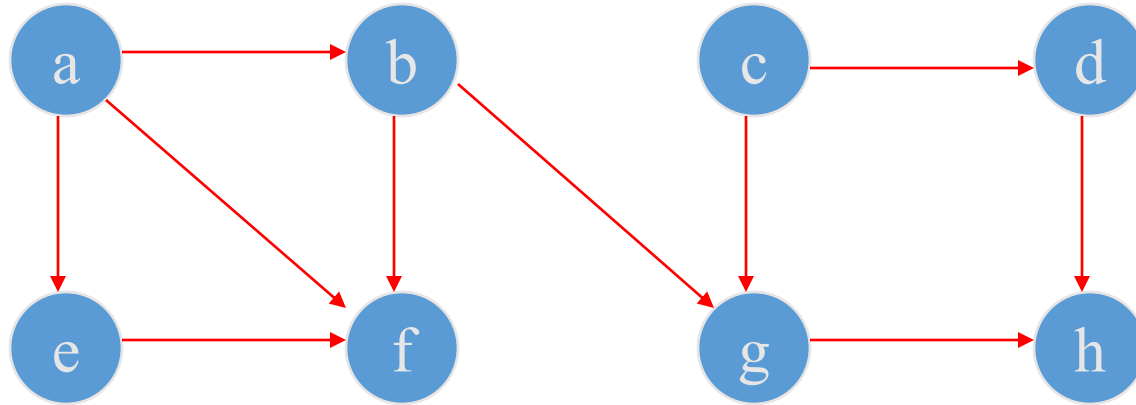
DFS-based algorithm for topological sorting

- Perform DFS traversal, noting the order vertices are popped off the traversal stack
- Reverse order solves topological sorting problem
- Back edges encountered? → NOT a dag!

Example:



DFS-based Algorithm



DFS Traversal queue

f_1 h_2

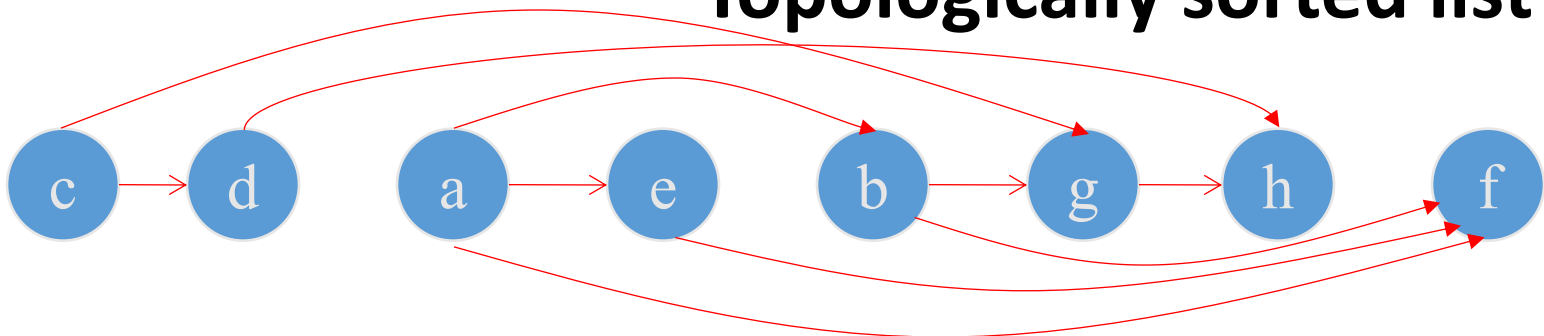
b_4 g_3 d_7

a_6 e_5 c_8

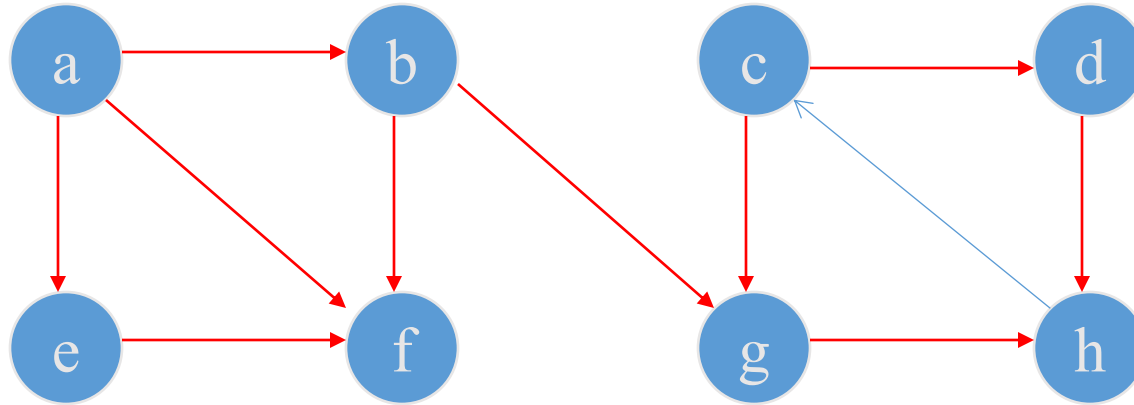
Popping-off order

f h g b e a d c

Topologically sorted list



DFS-based Algorithm



DFS Traversal queue

d back edge, not dead end.

c

f_1 h

b g

a

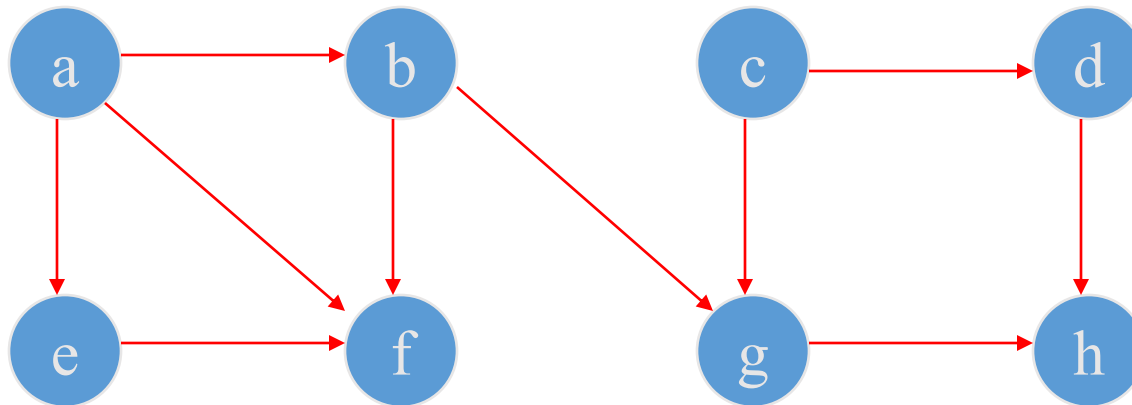
NO Topologically sorted list

Source Removal Algorithm

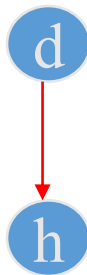
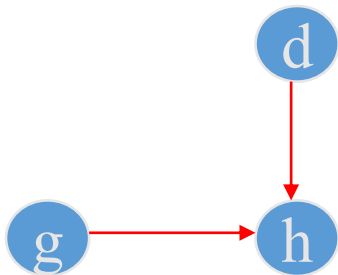
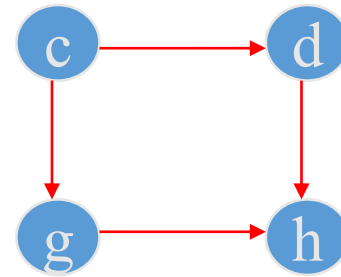
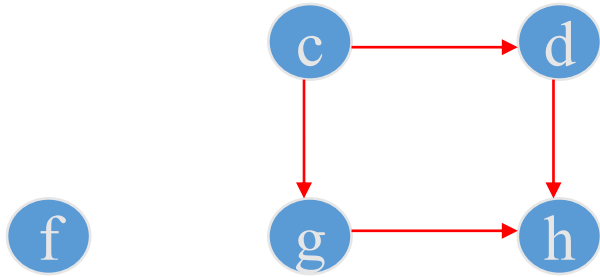
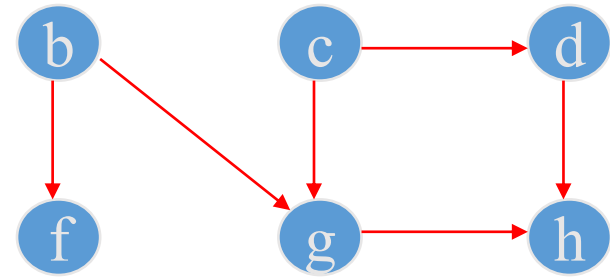
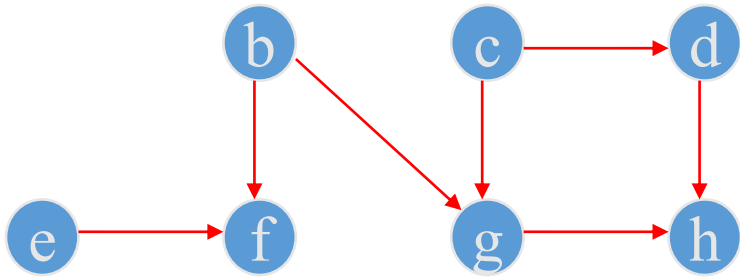
Source removal algorithm

Repeatedly identify and remove a *source* (a vertex with no incoming edges) and all the edges incident to it until either no vertex is left (problem is solved) or there is no source among remaining vertices (not a dag)

Example: Efficiency: same as efficiency of the DFS-based algorithm



Source Removal Algorithm



Course Registration

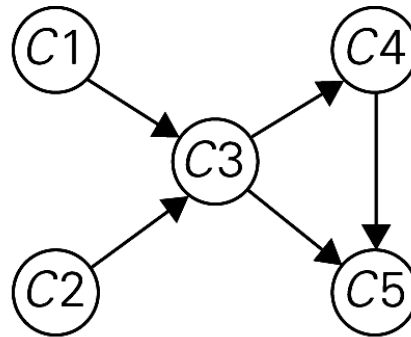
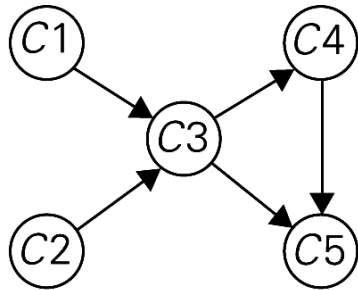


FIGURE 5.9 Digraph representing the prerequisite structure of five courses

Course Registration



(a)

$C5_1$
 $C4_2$
 $C3_3$
 $C1_4$ $C2_5$

(b)

The popping-off order:

$C5, C4, C3, C1, C2$

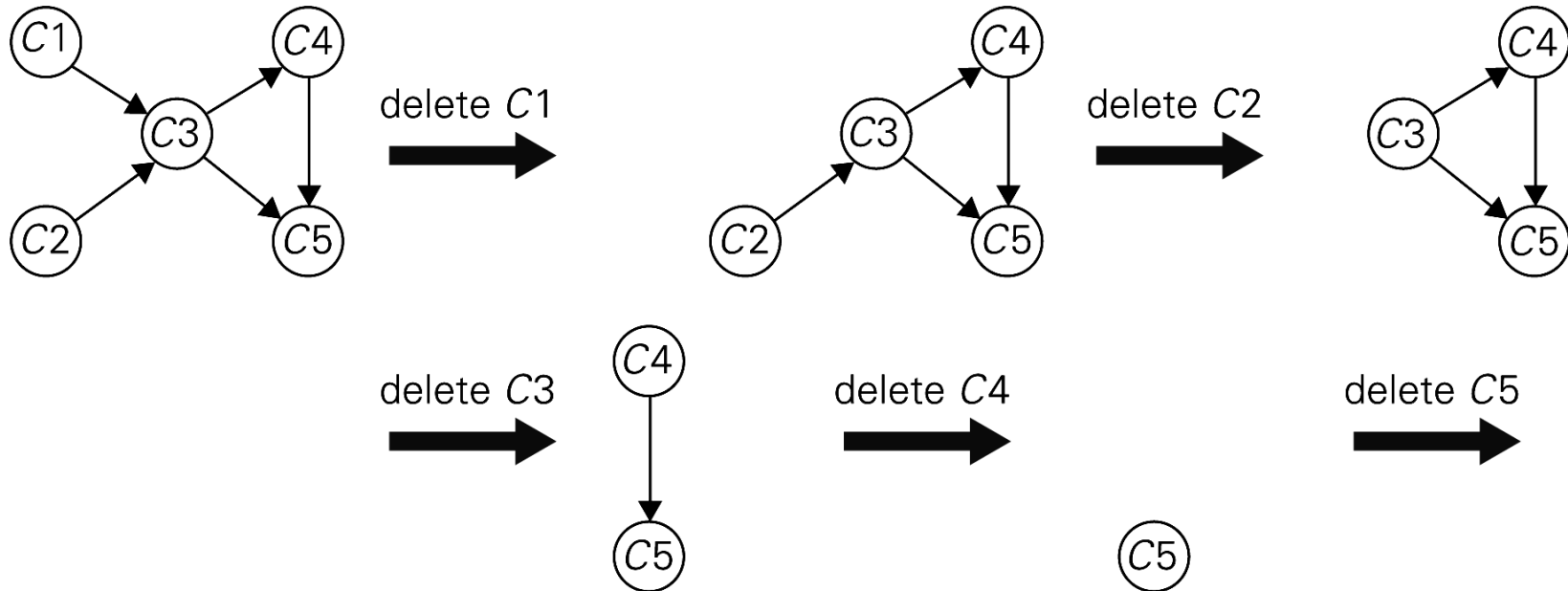
The topologically sorted list:

$C2 \rightarrow C1 \rightarrow C3 \rightarrow C4 \rightarrow C5$

(c)

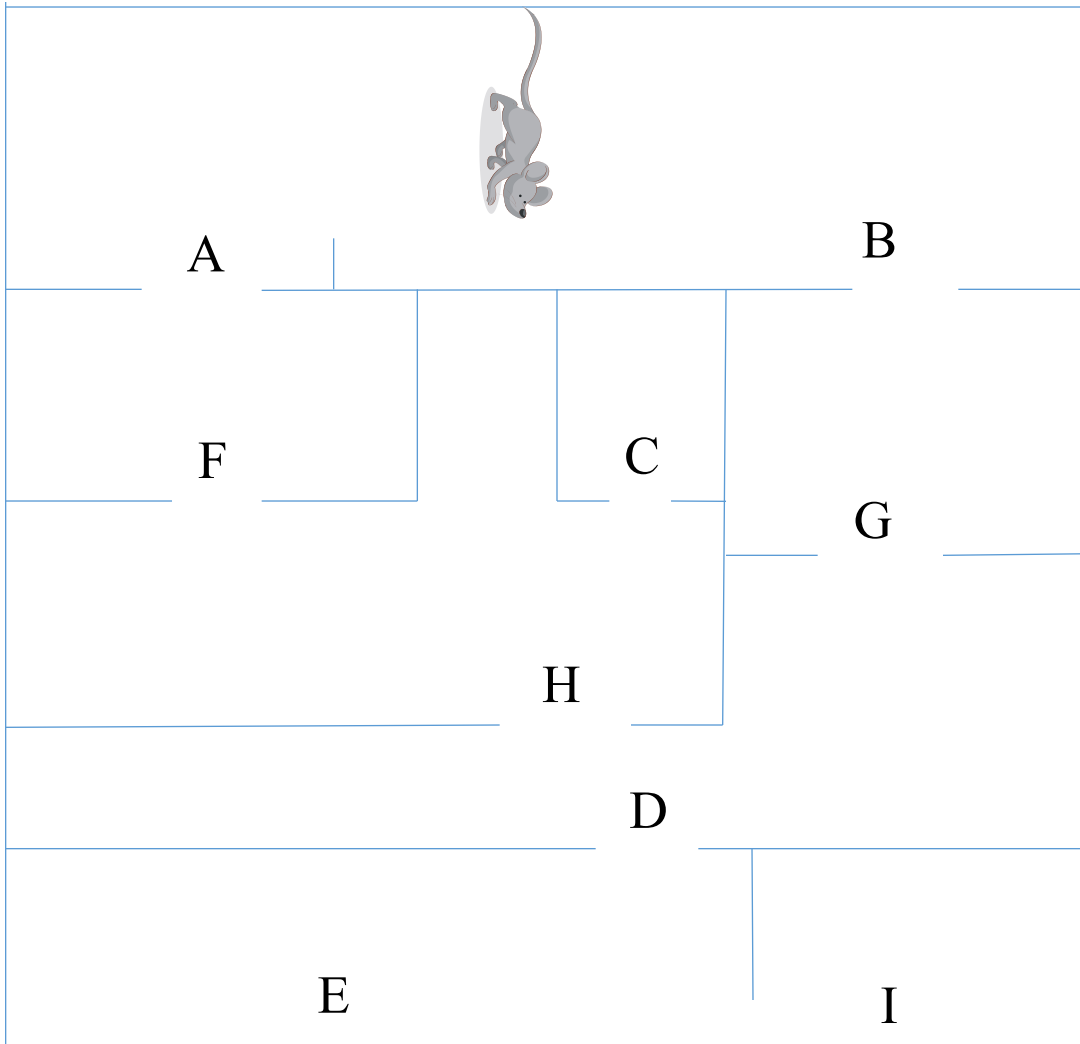
FIGURE 5.10 (a) Digraph for which the topological sorting problem needs to be solved. (b) DFS traversal stack with the subscript numbers indicating the popping-off order. (c) Solution to the problem.

Course Registration



The solution obtained is C1, C2, C3, C4, C5

FIGURE 5.11 Illustration of the source-removal algorithm for the topological sorting problem. On each iteration, a vertex with no incoming edges is deleted from the digraph.



- Suppose there is a trap at each door.
- Mouse runs down only.
- Which way is the lower risk?
- Which way is the high risk?
- Can we use topological sort if mouse can run in any direction?

Algorithms for Generating Combinatorial Objects

- Generating Permutations

Algorithm I

- Assuming that that smaller problem is solved. - The smaller-by-one problem generate all $(n-1)$ permutations.
- A solution of the larger problem is by inserting n in each of n possible positions among elements of every permutation of $(n-1)$ elements

Generating Permutations

Algorithm I

1

21

12

321 231 213 312 132 123

4321 3421 3241 3214 4231 2431 2341 2314

4213 2413 2143 2134 4312 3412 3142 3124

4132 1432 1342 1324 4123 1423 1243 1234

n	permutations
1	1
2	$2*1$
3	$3*2$
4	$4*6$

Generating Permutations

Algorithm I with **minimal change**

- Each permutation can be obtained from its immediate predecessor by **exchanging just two elements** in it.
 - Start by inserting n from right-to-left
 - Switch direction every time for a new permutation

(Right-to-Left, Left-to-Right, R-to-L, L-to-R, ...)

Generating Permutations

start	1		
insert 2 into 1 right to left	12	21	
insert 3 into 21 right to left	123	132	312
insert 3 into 21 left to right	321	231	213

FIGURE 5.12 Generating permutations bottom up

Generating Permutations

Algorithm I with **minimal change**

- Minimal change requirement is beneficial both algorithm's speed and for applications using permutations.
- Traveling salesman using exhaustive search can be generated by a minimal change algorithm, we can compute the length of a new tour from its predecessor.
[Computing constant rather than a linear time]

Algorithms for Generating Combinatorial Objects

- Generating Permutations

Algorithm II Johnson-Trotter

- Creating permutations of n elements **without** explicitly **generating permutations for smaller values of n .**
- Element k is said to be **mobile** if its an arrow points to a smaller number adjacent to it.

Generating Permutations

- JohnsonTrotter(n)

initialize the first permutation of with 1 2 ...n



while the last permutation has mobile **do**

 find its largest mobile element k

 swap k and adjacent integer k's arrow point to
 reverse direction of all element that are larger
 than k

 add the new permutation to the list.

Generating Permutations

- JohnsonTrotter(n)

initialize the first permutation of with 1 2 ...n

while the last permutation has mobile **do**

find its largest mobile element k

swap k and adjacent integer k's arrow point to
reverse direction of all element that are larger
than k

add the new permutation to the list.

←←←	←←←	←←←	→←←	←→←	←←→
1 2 3	1 3 2	3 1 2	3 2 1	2 3 1	2 1 3

n	subsets								
0	$\emptyset$								
1	$\emptyset$	$\{a_1\}$							
2	$\emptyset$	$\{a_1\}$	$\{a_2\}$	$\{a_1, a_2\}$					
3	$\emptyset$	$\{a_1\}$	$\{a_2\}$	$\{a_1, a_2\}$	$\{a_3\}$	$\{a_1, a_3\}$	$\{a_2, a_3\}$	$\{a_1, a_2, a_3\}$	

FIGURE 5.13 Generating subsets bottom up

Decrease-by-Constant-Factor Algorithms

In this variation of decrease-and-conquer, instance size is reduced by the same factor (typically, 2)

Examples:

- Binary search and the method of bisection
- Exponentiation by squaring
- Multiplication à la russe (Russian peasant method)
- Fake-coin puzzle
- Josephus problem

Exponentiation by Squaring

The problem: Compute a^n where n is a nonnegative integer. The problem can be solved by applying recursively the formulas:

For even values of n

$$a^n = (a^{n/2})^2 \text{ if } n > 0 \text{ and } a^0 = 1$$

For odd values of n

$$a^n = (a^{(n-1)/2})^2 a$$

Recurrence: $M(n) = M(\lfloor n/2 \rfloor) + f(n)$, where $f(n) = 1$ or 2 ,

$$M(0) = 0$$

Master Theorem: $M(n) \in \Theta(\log n) = \Theta(b)$ where $b = \lceil \log_2(n+1) \rceil$

Russian Peasant Multiplication

The problem: Compute the product of two positive integers. Can be solved by a decrease-by-half algorithm based on the following formulas.

For even values of n :

$$n * m = \frac{n * 2m}{2}$$

For odd values of n :

$$n * m = \frac{n-1 * 2m}{2} + m \text{ if } n > 1 \text{ and } m \text{ if } n = 1$$

Example of Russian Peasant Multiplication

Compute $20 * 26$

n	m
-----	-----

20	26
----	----

10	52
----	----

5	104
---	-----

2	208	(+ 104)
---	-----	---------

1	416	<u>+104</u> = 520
---	-----	-------------------

Note: Method reduces to adding m 's values corresponding to odd n 's.

Fake-Coin Puzzle (simpler version)

There are n identically looking coins one of which is fake.

There is a balance scale but there are no weights; the scale can tell whether two sets of coins weigh the same and, if not, which of the two sets is heavier (but not by how much). Design an efficient algorithm for detecting the fake coin. Assume that the fake coin is known to be lighter than the genuine ones.

Decrease by factor 2 algorithm

Decrease by factor 3 algorithm

Josephus Problem

-

Josephus Problem

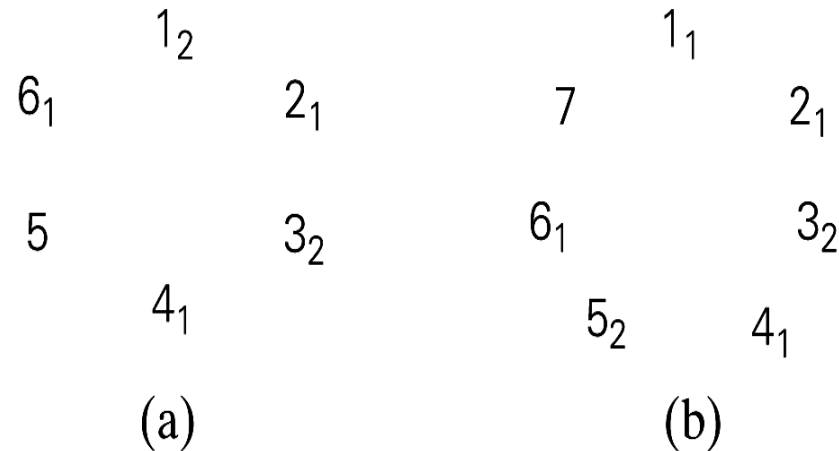


FIGURE 5.15 Instances of the Josephus problem for (a) $n = 6$ and (b) $n = 7$. Subscript numbers indicate the pass on which the person in that position is eliminated. The solutions are $J(6) = 5$ and $J(7) = 7$, respectively.

Josephus Problem

$$J(1) = 1$$

$$J(2k) = 2J(k) - 1 \quad \text{case for even } n = 2k$$

$$J(2k+1) = 2J(k) + 1 \quad \text{case for odd } n = 2k+1$$

$$J(n) = ? \quad \text{without recursion}$$

Homework

- Exercise 4.4 on page 182
Questions 11, 12, 13

Variable-Size-Decrease Algorithms

In the variable-size-decrease variation of decrease-and-conquer, instance size reduction varies from one iteration to another

Examples:

- Euclid's algorithm for greatest common divisor
- Partition-based algorithm for selection problem
- Interpolation search
- Some algorithms on binary search trees
- Nim and Nim-like games

Euclid's Algorithm

Euclid's algorithm is based on repeated application of equality

$$\gcd(m, n) = \gcd(n, m \bmod n)$$

$$\text{Ex.: } \gcd(80, 44) = \gcd(44, 36) = \gcd(36, 8) = \gcd(8, 4) = \gcd(4, 0) = 4$$

One can prove that the size, measured by the second number, decreases at least by half after two consecutive iterations.

Hence, $T(n) \in O(\log n)$

Selection Problem

Find the k^{th} smallest element in a list of n numbers

- $k = 1$ or $k = n$
- median: $k = \lceil n/2 \rceil$

Example: 4, 1, 10, 9, 7, 12, 8, 2, 15 median =
?

The median is used in statistics as a measure of an average value of a sample. In fact, it is a better (more robust) indicator than the mean, which is used for the same purpose.

Digression: Post Office Location Problem

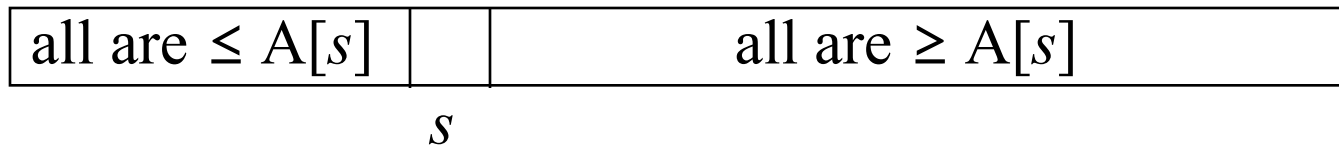
Given n village locations along a straight highway, where should a new post office be located to minimize the average distance from the villages to the post office?

Algorithms for the Selection Problem

The sorting-based algorithm: Sort and return the k -th element

Efficiency (if sorted by mergesort): $\Theta(n \log n)$

A faster algorithm is based on using the quicksort-like partition of the list. Let s be a split position obtained by a partition:



Assuming that the list is indexed from 1 to n :

If $s = k$, the problem is solved;

if $s > k$, look for the k -th smallest elem. in the left part;

if $s < k$, look for the $(k-s)$ -th smallest elem. in the right part.

Note: The algorithm can simply continue until $s = k$.

Tracing the Median / Selection Algorithm

Example: 4 1 10 9 7 12 8 2 15 Here: $n = 9$, $k = \lceil 9/2 \rceil = 5$

array index 1 2 3 4 5 6 7 8 9

4 1 10 9 7 12 8 2 15

4 1 2 9 7 12 8 10 15

2 1 4 9 7 12 8 10 15 --- $s=3 < k=5$

9 7 12 8 10 15

9 7 8 12 10 15

 8 7 9 12 10 15 --- $s=6 > k=5$

8 7

 7 8 --- $s=k=5$

Solution: median is 8

Selection Problem

4, 1, 10, 9, 7, 12, 8, 2, 15

2, 1, **4**, 9, 7, 12, 8, 10, 15

$s=3 < k=5$

9, 7, 12, 8, 10, 15

8, 7, **9**, 12, 10, 15

$s=6 < k=5$

8, 7

7, **8**

$s=5=k$ hence we can stop: the median is 8.

Efficiency of the Partition-based Algorithm

Average case (average split in the middle):

$$C(n) = C(n/2) + (n+1) \qquad C(n) \in \Theta(n)$$

Worst case (degenerate split): $C(n) \in \Theta(n^2)$

A more sophisticated choice of the pivot leads to a complicated algorithm with $\Theta(n)$ worst-case efficiency.

Efficiency of the Partition-based Algorithm

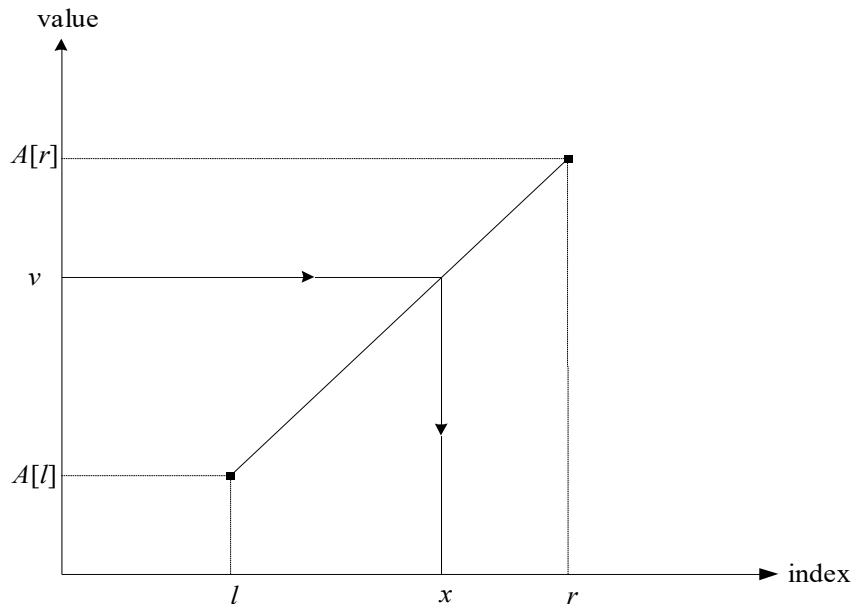
- If we sort the array first, and retrieve by position, we efficiency is $O(n \log n)$
- If we partition and choose only one partition as we did, the recurrence for the number of comparisons is

$$C(n) = C(n/2) + (n+1)$$

Using Master Theorem, the efficiency is $\Theta(n)$.

Interpolation Search

Searches a sorted array similar to binary search but estimates location of the search key in $A[l..r]$ by using its value v . Specifically, the values of the array's elements are assumed to grow linearly from $A[l]$ to $A[r]$ and the location of v is estimated as the x -coordinate of the point on the straight line through $(l, A[l])$ and $(r, A[r])$ whose y -coordinate is v :



$$x = l + \left\lfloor \frac{(v - A[l]) (r - l)}{(A[r] - A[l])} \right\rfloor$$

Analysis of Interpolation Search

- Efficiency

average case: $C(n) < \log_2 \log_2 n + 1$

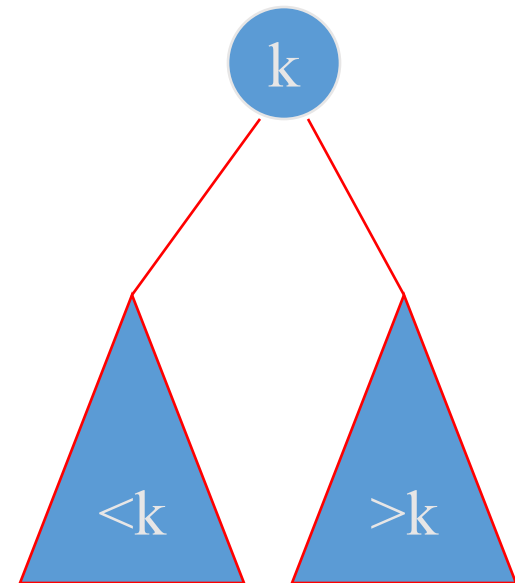
worst case: $C(n) = n$

- Preferable to binary search only for VERY large arrays and/or expensive comparisons
- Has a counterpart, the method of false position (regula falsi), for solving equations in one unknown (Sec. 12.4)

Binary Search Tree Algorithms

Several algorithms on BST requires recursive processing of just one of its subtrees, e.g.,

- Searching
- Insertion of a new key
- Finding the smallest
(or the largest) key



Searching in Binary Search Tree

Algorithm *BTS*(x, v)

//Searches for node with key equal to v in BST rooted at node x

if $x = \text{NULL}$ return -1

else if $v = K(x)$ return x

else if $v < K(x)$ return *BTS*(*left*(x), v)

else return *BTS*(*right*(x), v)

Efficiency

worst case: $C(n) = n$

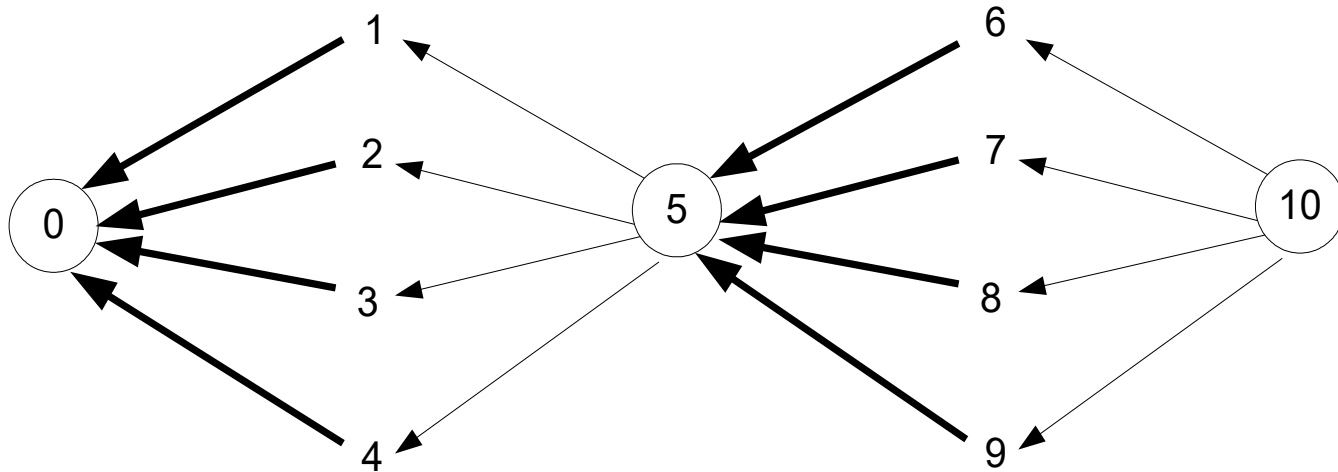
average case: $C(n) \approx 2 \ln n \approx 1.39 \log_2 n$

One-Pile Nim

There is a pile of n chips. Two players take turn by removing from the pile at least 1 and at most m chips. (The number of chips taken can vary from move to move.) The winner is the player that takes the last chip. Who wins the game – the player moving first or second, if both player make the best moves possible?

It's a good idea to analyze this and similar games “backwards”, i.e., starting with $n = 0, 1, 2, \dots$

Partial Graph of One-Pile Nim with $m = 4$



Vertex numbers indicate n , the number of chips in the pile. The losing position for the player to move are circled. Only winning moves from a winning position are shown (in bold).

Generalization: The player moving first wins iff n is not a multiple of 5 (more generally, $m+1$); the winning move is to take $n \bmod 5$ ($n \bmod (m+1)$) chips on every move.

Game of Nim

- Obsolete English verb *nim* means “take”
- *NIM* upside-down and backwards results in *WIN*
- Similar game of One-pile Nim is a summation game: a player picks a positive integer less than, say, ten and then his opponent and he take turns adding any integer less than ten; the first player to reach 100 exactly is the winner.

Homework

- Exercise 4.5 on page 187
Questions 3, and 10 (*Mis`ere one-pile Nim*)